

What is an http server ?

Pour faire simple, un **serveur HTTP** est un logiciel (ou l'ordinateur qui l'héberge) dont le rôle est de distribuer des pages web aux utilisateurs.

C'est le "bibliothécaire" du Web : vous lui demandez un livre (une page), il va le chercher dans ses rayons (ses dossiers) et vous le tend.

Comment ça fonctionne ? (Le modèle Client-Serveur)

Le fonctionnement repose sur un dialogue structuré entre deux acteurs :

1. **Le Client (Vous)** : Votre navigateur (Chrome, Firefox, LibreOffice pour certains services en ligne) envoie une **Requête HTTP**.
2. **Le Serveur** : Il reçoit la requête, vérifie si le fichier existe, et renvoie une **Réponse HTTP** contenant le code de la page (HTML, CSS, images).

Les serveurs HTTP les plus connus

Si vous travaillez sous Ubuntu, vous rencontrerez souvent ces trois-là :

- **Apache** : Le plus ancien et le plus modulable (très utilisé avec OpenOffice/LibreOffice pour les serveurs de documents).
 - **Nginx** : Connu pour sa rapidité et sa légèreté.
 - **Node.js** : Très populaire pour les applications web modernes en temps réel.
-

Anatomie d'une requête

Quand vous tapez une adresse, le serveur interprète plusieurs informations :

- **L'URL** : L'adresse précise de la ressource.
- **La Méthode** : * GET : "Donne-moi cette page."
 - POST : "Envoie ces données" (comme un formulaire d'inscription).
- **Le Code de statut** : La réponse du serveur.
 - 200 OK : Tout s'est bien passé.
 - 404 Not Found : La page n'existe pas.

Pourquoi est-ce utile pour vous ?

Même si vous utilisez principalement LibreOffice, comprendre le HTTP est utile si :

- Vous voulez partager des documents sur un réseau local.
- Vous utilisez des extensions qui se connectent à des bases de données distantes.
- Vous souhaitez installer votre propre "Cloud" personnel (comme Nextcloud) sur Ubuntu.

Avantages / inconvénients des mécanismes évènementiels `poll()`, `select()`, `epoll` ?

Lorsqu'on développe un serveur HTTP ou n'importe quelle application réseau sous Linux (comme Ubuntu), on est confronté au problème de la **gestion de multiples connexions simultanées**.

Plutôt que de créer un processus par client (ce qui consomme énormément de RAM), on utilise le **multiplexage d'E/S**. Voici la comparaison entre les trois mécanismes historiques.

1. `select()` : L'ancêtre universel

C'est le plus vieux (POSIX). Il surveille trois ensembles de descripteurs de fichiers (lecture, écriture, erreurs).

- **Avantages :**
 - **Ultra-portable :** Disponible sur quasiment tous les systèmes (Unix, Windows, macOS).
 - Précision du timeout plus fine sur certains vieux systèmes.
- **Inconvénients :**
 - **Limite de descripteurs :** Souvent limité à **1024** (FD_SETSIZE).
 - **Inefficace :** À chaque appel, le noyau doit scanner toute la liste pour voir qui est prêt. C'est une complexité en $O(n)$.
 - L'application doit reconstruire la liste à chaque itération car `select` la modifie.

2. `poll()` : Le successeur plus souple

Arrivé plus tard, il corrige la limitation principale de `select`.

- **Avantages :**
 - **Pas de limite fixe :** On peut surveiller autant de fichiers que le système le permet.
 - Structure de données plus propre (`pollfd`).
- **Inconvénients :**
 - **Toujours $O(n)$:** Comme `select`, plus vous avez de clients, plus c'est lent. Si vous avez 10 000 clients et qu'un seul parle, le système doit quand même vérifier les 10 000.
 - Peu performant pour les serveurs à très haute charge.

3. `epoll()` : Le champion de Linux

C'est l'outil spécifique à Linux (depuis le noyau 2.6) conçu pour le "C10k problem" (gérer 10 000 connexions en même temps).

- **Avantages :**

- **Efficacité maximale** : Complexité en $O(1)$. Le noyau maintient une liste des descripteurs actifs. L'application ne reçoit que les événements qui ont réellement eu lieu.
- **Persistance** : On n'a pas besoin de redonner la liste complète des fichiers à surveiller à chaque appel.
- Supporte le mode **Edge-Triggered** (ne prévient qu'une fois lors d'un changement d'état), ce qui est encore plus rapide.
- **Inconvénients** :
 - **Non portable** : Spécifique à Linux (sur FreeBSD/macOS, on utilise `kqueue`).

Est-ce plus simple d'utiliser `select()` par rapport à `Poll` ou `epoll`?

Gemini a dit

La réponse courte est : **oui et non**. Tout dépend de ce que tu appelles "simple" : est-ce la simplicité d'écriture, la simplicité de compréhension ou la simplicité de portabilité ?

Voici l'analyse comparative pour t'aider à choisir :

1. Pourquoi `select()` peut paraître plus "simple"

- **Universalité** : C'est la méthode historique. Si tu apprends le réseau à l'école ou via de vieux tutoriels, c'est souvent le premier exemple. Il fonctionne partout (Windows, Linux, macOS).
- **Timeout précis** : La structure `struct timeval` permet de descendre à la microseconde, ce qui est parfois plus intuitif pour certains développeurs que les millisecondes de `poll()`.

2. Pourquoi `select()` est en fait "plus compliqué" à l'usage

Paradoxalement, `select()` demande plus de code de "gestion" que ses successeurs :

- **Destruction de la liste** : `select()` modifie l'ensemble de descripteurs (`fd_set`) que tu lui passes. Cela signifie que tu dois **reconstruire ta liste à chaque itération** de ta boucle `while(1)`.
 - **Gestion des index** : Tu dois manuellement garder une trace du plus grand descripteur de fichier (`nfds`) pour le passer en argument, sinon le noyau ne scanne pas tout.
 - **Macros rigides** : Tu dois utiliser des macros spécifiques (`FD_ZERO`, `FD_SET`, `FD_ISSET`) qui sont parfois moins lisibles qu'une simple structure de données.
-

3. Pourquoi `poll()` est souvent considéré comme le plus "équilibré"

Pour un débutant sous Ubuntu (Linux), `poll()` est généralement plus simple à coder que `select()` :

- **Structure claire :** Tu utilises un tableau de structures `pollfd`. Tu n'as pas besoin de reconstruire tout le tableau à chaque fois, il suffit de lire le champ `revents` pour savoir ce qui s'est passé.
- **Pas de limite arbitraire :** Tu n'as pas à te soucier de la limite des 1024 fichiers.

4. Et `epoll()` ?

`epoll()` est techniquement le plus complexe à mettre en œuvre car il nécessite trois appels système différents (`epoll_create`, `epoll_ctl`, `epoll_wait`). Il est réservé aux performances extrêmes.

Ask if they use only one `poll()` (or equivalent) and how they handle server `accept()` and client `read/write` ?

Only one `select()` use in `SelectMultiplexer::wait()`, called into `ServerManager::run()` ; `accept()` into `Socket::acceptSocket()` called into `ConnectionManager::addClient()`.

→ `accept()` : to handle new client connections

via `fcntl()`, which changed the socket from blocking to non blocking mode.

Here is the breakdown of why this is critical for your server.

1. The Problem: Blocking I/O

By default, sockets operate in "blocking" mode.

- **Reading:** If you call `recv()` or `read()` on a blocking socket and the client hasn't sent any data yet, your server will **stop and wait (sleep)** until data arrives.
- **Writing:** If you call `send()` and the kernel's write buffer is full, your server will **stop and wait** until there is space.

The Consequence: Since webserv is usually a single-threaded, event-driven server (using `poll`, `select`, or `epoll`), if the code blocks waiting for Client A, **the entire server freezes**. It cannot accept new connections or process requests for Client B until Client A does something.

2. The Solution: `fcntl()` with `O_NONBLOCK`

You use `fcntl` to change the flags of the file descriptor to enable **Non-Blocking I/O**.

After `poll()` (or equivalent), I/O on event-driven descriptors (e.g., listening/client sockets, only when readiness is indicated. It is acceptable to read/write until `EAGAIN/EWOULDBLOCK` main loop. Ask the group to show the code path from `poll()` (or equivalent) to the client's read and write.

See `serverManager::run()`

On almost all modern operating systems (Linux, macOS, FreeBSDs), **`EAGAIN/EWOULDBLOCK`** are defined as the same numeric value.

This means that to the compiler, they are identical. Checking for one is mathematically the same as checking for the other.

2. The History (Why do we have two?)

The existence of two names for the same error is a relic of the "Unix Wars" in the 1980s between two main branches of Unix:

- **BSD (Berkeley Unix):** They introduced Sockets. When a non-blocking socket couldn't send/receive, they returned **EWOULDBLOCK** (meaning: "Operation would block if you weren't non-blocking").
- **System V (AT&T Unix):** They focused on general file/pipe I/O. When a non-blocking file was busy, they returned **EAGAIN** (meaning: "Try again later").

When the POSIX standard was created to unify these operating systems, it had to support legacy code from both sides, so it kept both names.

3. Which one should you use?

Even though they are likely the same value on your machine, **portability best practices** dictate that you should check for both. This protects you in case your code ever runs on an obscure or ancient system where they are different.

Search for all read/recv/write/send on a socket and check that, if an error is returned, the client is removed.

Search for all read/recv/write/send and check if the returned value is correctly handled (checking only -1 or 0 is not enough; both must be handled).

send() == 0: You effectively sent nothing (usually because you asked to send 0 bytes). It does NOT mean the client disconnected. When there is nothing to send, better to avoid calling send !

If errno is checked after read/recv/write/send, the grade is 0 and the evaluation process ends
→ logged, not used to drive control flow.

Writing or reading event-driven descriptors (sockets, pipes/FIFOs, TTY, etc.) without going through select or equivalent is strictly FORBIDDEN. Regular disk files are exempt.

→ select() into SelectMultiplexer::wait()

Confirm that synchronous I/O on regular disk files (e.g., reading the configuration or sma event loop.

?

The project must compile without any relinking issues. If not, use the 'Invalid compilation' flag
If any point is unclear or is not correct, the evaluation stops.

Configuration

In the configuration file, check whether you can do the following and test the result:

- *Search for the HTTP response status codes list on the internet. During this evaluation, if any status codes is wrong, do not award any points related to them.*

OK

- *Set up multiple websites on different interfaces and ports.*

OK

- Set up a default error page (try modifying the 404 error page).

OK

- Limit the size of the client request body (use: curl -X POST -H "Content-Type: plain/text -data « BODY IS HERE write something shorter or longer than body limit").

OK

- Set up routes on a server to different directories.

OK

- Set up a default file to serve when requesting a directory.

OK

- Set up a list of accepted methods for a specific route (e.g., try DELETE something with and without permission)

OK

Basic checks

Using telnet, curl, and pre-prepared files, demonstrate that the following features function correctly:

- GET, POST, and DELETE requests should work.

OK

- UNKNOWN requests should not result in a crash.

OK

- For every test, you should receive the appropriate status code.

OK

- Upload some files to the server and retrieve them.

OK

Check CGI

Pay attention to the following:

- The server should function correctly with CGI.

OK

- The CGI should be run in the correct directory for relative path file access.

OK

- With the help of the student(s), you should check that everything is working properly. You "GET" and "POST" methods.

OK

- You must test with CGI files containing errors to ensure that server's error handling work containing an infinite loop or an error; you are free to do whatever tests you want within t remain at your discretion. The group being evaluated should help you with this.

OK

- The server should never crash, and an error should be displayed if an issue occurs.

OK

Check with a browser

- Use the browser chosen by the team. Open the network tab and try connecting to the server with it

OK

- Look at the request header and response header.

OK

- It should be compatible with serving a fully static website.

OK

- Try an incorrect URL on the server.

OK

- Try to list a directory.

OK

- Try a redirected URL.

OK

- Try anything you like.

OK

Port issues

- In the configuration file, set up multiple interfaces and ports to provide different websites, use the browser to check that the configuration works correctly and serves the appropriate website.

OK

- Configure multiple websites on the same interface:port. This should result in an error, unless implement the virtual host feature. Both approaches are valid, as long as everything work

OK

- Launch multiple webserv programs at the same time with different configuration files but with common ports : Does it work? If it does, ask why. Ensure that, whatever the group's choice was, the program does not crash.

OK

Siege & stress test

- Use Siege to run some stress tests.

OK

- Availability should be above 99.5% for a simple GET request on an empty page using a siege -b on that page

OK

- Ensure there are no memory leaks (Monitor the process memory usage; it should not increase indefinitely)

OK

- Check that there are no hanging connections.

You should be able to use siege indefinitely without having to restart the server (take a look at siege -b)

OK

When conducting load tests using the siege command, be careful, it depends on your OS. of connections per second by specifying options such as -c (number of clients), -d (maxim reconnects), and -r (number of attempts). The choice of these parameters is at the evaluat imperative to reach an agreement with the person being evaluated to ensure a fair and tran server's performance.

OK

Cookies and session

There is a functional session and cookie system on the web server.

CGI

The web server supports multiple CGI systems.